

HIPAA FOR MEDICAL APPLICATION DEVELOPERS

A practical engineering handbook and workbook

Architecture • Code • Evidence • Clinical Integrations • Secure Delivery

Anthony Narine
Developer learning edition | September 2026

Copyright © 2026 Anthony Narine. All rights reserved.

This book is an educational engineering handbook. It is not legal advice, a compliance certification, or a substitute for counsel, a privacy officer, a security officer, or organization-specific policy. HIPAA obligations depend on role, data flow, contracts, purpose, and facts. The HIPAA regulations and official HHS guidance control if this book conflicts with them.

Regulatory status used in this edition: HHS states that the existing HIPAA Security Rule remains the rule currently in effect. HHS issued a proposed rule in late 2024 to strengthen cybersecurity requirements; a proposed rule is not a final rule. Re-check HHS and eCFR before making production or contractual decisions.

The engineering promise of this book

By the end, you should be able to look at a medical application and answer five questions: What regulated data exists? Where does it move? Who is allowed to act on it? What can go wrong? What evidence proves the control actually works?

How to use this handbook

This is not a glossary with code sprinkled on top. It is a build-and-review method. HIPAA tells regulated organizations what outcomes must be achieved. Your job as a developer is to translate those outcomes into trust boundaries, data contracts, authorization decisions, durable evidence, recovery behavior, vendor boundaries, and tests.

Lumen, a vascular ultrasound reporting platform, is the primary case study because it is rich enough to make the hard parts real: patient identity, measurements, conclusions, signed reports, PDFs, DICOM images, HL7 interfaces, multi-tenant organizations, facilities, cloud storage, and AI-assisted workflows. Gait is used for identity and security examples. EstateIQ appears occasionally as a boundary example because private data is not automatically PHI.

Recurring engineering loop

RULE → DEVELOPER QUESTION → ARCHITECTURE → CODE ENFORCEMENT
POINT → NEGATIVE TEST → EVIDENCE → GAP / RESIDUAL RISK

What this book covers

- HIPAA foundations: Privacy, Security, Breach Notification, HITECH, covered entities, business associates, PHI and ePHI.
- Data-use engineering: minimum necessary, de-identification, Safe Harbor, Expert Determination, limited data sets, DUAs, AI boundaries.
- Security Rule implementation: risk analysis, access control, audit controls, integrity, authentication, transmission security, availability, vendors and documentation.
- Medical integrations: HL7, MLLP, TCP, DICOM, object storage, cloud services, clinical workflow integrity and downtime behavior.
- Modern application security: OWASP Top 10:2025, OWASP ASVS 5.0, threat modeling, secure SDLC, supply chain, error handling, testing and incident response.

Primary sources for this edition include 45 CFR Parts 160 and 164, HHS/OCR HIPAA guidance, and official OWASP Top 10:2025 / ASVS 5.0 materials. See the Reference Map.

Contents

PART I — Understand the law before the code

1. What HIPAA is - and is not
2. Covered entities, business associates, and the BAA perimeter
3. PHI, ePHI, identifiers, and context
4. Privacy Rule vs Security Rule vs Breach Notification Rule
5. How to read HIPAA like an engineer

PART II — Control the data

6. Minimum necessary as API and query design
7. De-identification: the boundary where PHI can stop being PHI
8. Safe Harbor in practice
9. Expert Determination and the Limited Data Set
10. Free text, analytics, research, and AI model boundaries

PART III — Build the Security Rule into architecture

11. Risk analysis and risk management
12. Administrative and physical safeguards developers still need to understand
13. Access control and emergency access
14. Authentication, sessions, and identity proof
15. Audit controls: security telemetry vs ePHI activity
16. Integrity and clinical meaning
17. Transmission security and encryption decisions
18. Availability, backups, and downtime
19. Documentation, evidence, retention, and review

PART IV — Clinical systems and cloud boundaries

20. HL7, MLLP, TCP, and interface security
21. DICOM and medical image security
22. Cloud vendors, BAAs, secrets, and object storage
23. Multi-tenancy, facilities, support access, and break-glass

PART V — Secure software engineering

24. OWASP Top 10:2025 for medical applications
25. OWASP ASVS 5.0 as a verification backbone
26. Threat modeling and secure design
27. Secure development lifecycle and supply-chain assurance
28. Testing controls, exceptional conditions, and fail-safe behavior

29. Incident response, breach readiness, and evidence preservation

PART VI — Capstone

30. Perform a HIPAA engineering review of Lumen - then your own system

PART I

Understand the law before the code

Chapter 1 What HIPAA Is - and Is Not

Learning objective. Explain the role of HIPAA in a medical application without treating “HIPAA compliant” as a magic product property.

HIPAA is a federal legal framework, not a software framework. For developers, the most useful mental model is that HIPAA creates obligations around information, organizations, people, contracts, processes, and evidence. Code is one mechanism used to satisfy some of those obligations. A perfect access-control function cannot replace workforce training, a missing business associate agreement, or an incident-response process. Likewise, good policies cannot compensate for an API that lets one tenant enumerate another tenant’s exams.

Three rules you must keep separate

- Privacy Rule: governs many uses and disclosures of PHI and gives individuals rights concerning their information.
- Security Rule: protects ePHI through administrative, physical, and technical safeguards.
- Breach Notification Rule: governs notification after certain impermissible uses or disclosures that compromise PHI, subject to the rule’s definitions and risk-assessment framework.

HITECH strengthened HIPAA, including direct Security Rule obligations for business associates and expanded enforcement. Your architecture therefore sits inside a legal and organizational system; it is not the whole system.

Do not make this claim lightly

“HIPAA compliant” is not equivalent to “encrypted,” “hosted on AWS,” “uses a BAA,” “passed a penetration test,” or “uses a healthcare model.” Compliance is a system-of-controls conclusion that depends on the regulated entity’s full facts.

Rule	45 CFR Part 164
Developer question	Which HIPAA rule applies to this data flow and actor?
Architecture answer	Data classification + role/contract map + security/privacy controls.
Evidence	Current diagrams, contracts, policies, tests, audit evidence.

Mastery check

Why can a technically secure product still fail HIPAA?

Why can a product outside HIPAA still deserve strong privacy and security controls?

Chapter 2 Covered Entities, Business Associates, and the BAA Perimeter

Learning objective. Determine who is regulated in a software relationship and why contracts and data flow change architecture decisions.

The Security Rule applies to covered entities and business associates. A software vendor can become a business associate when it creates, receives, maintains, or transmits PHI on behalf of a covered entity for a regulated service. The same vendor can have one HIPAA-regulated customer deployment and another workflow that is outside HIPAA. Start with the relationship and purpose, not the file extension.

A developer-facing boundary test

1. Identify the customer and its HIPAA role.
2. Identify what information your service creates, receives, maintains, or transmits for that customer.
3. Identify every subcontractor that can receive or maintain that information.
4. Identify which services are included in contractual assurances such as BAAs.
5. Draw the “BAA perimeter”: which systems and vendors are approved to handle PHI for this workflow?

A BAA does not make unsafe code safe. It establishes contractual duties and permitted handling. Conversely, a strong technical stack does not remove the need for a BAA when one is required.

Lumen example

If Lumen stores patient identifiers, vascular measurements, images, and reports for a hospital, the deployment is plainly handling PHI/ePHI. If an external error-tracking service receives report payloads, that service has crossed into the regulated data flow whether or not the developer intended it.

Exercise: Draw your BAA perimeter

Draw Lumen, its database, media storage, identity provider, logging provider, AI provider, email/SMS services, and hospital interface engine. Mark which components can receive PHI and which contractual boundary covers each one.

Chapter 3 PHI, ePHI, Identifiers, and Context

Learning objective. Classify information correctly and explain why identifiers become regulated through context and linkage.

PHI is not synonymous with “medical-looking data.” HIPAA protects individually identifiable health information in covered contexts. ePHI is the electronic subset protected by the Security Rule. A blood-flow velocity with no person or context may be ordinary scientific data; the same value tied to a patient encounter is part of a health record.

Think in data classes, not database tables

- Direct identifiers: name, MRN, email, phone, account identifiers and similar direct links.
- Clinical content: diagnoses, measurements, conclusions, imaging, medications, procedures, notes.
- Operational metadata: timestamps, facility, clinician, device, interface destination, request identifiers.
- Security metadata: subject IDs, IP addresses, session IDs, denied actions, authentication events.
- Derived data: scores, model embeddings, labels, summaries, feature vectors, and analytics outputs.

Do not assume derived data is outside HIPAA merely because the original name is absent. If the data can still be linked to an individual in the regulated context, it may remain PHI.

EstateIQ boundary example

A landlord’s tenant name and rent ledger are private data, but they are not PHI merely because they are sensitive. HIPAA is relationship- and purpose-dependent.

Mastery check

Give one example of data that looks harmless until it is linked to an encounter.

What is the difference between PHI and ePHI for Security Rule scope?

Chapter 4 Privacy Rule vs Security Rule vs Breach Notification Rule

Learning objective. Separate data-use permission, security safeguards, and breach response so architecture discussions do not collapse three different problems into one.

A useful engineering shorthand is: the Privacy Rule asks when PHI may be used or disclosed; the Security Rule asks how ePHI must be protected; the Breach Notification Rule asks what happens after certain impermissible uses or disclosures. This is simplified, but it prevents a common design failure: treating security controls as permission to use data for any purpose.

```
Privacy question: May this PHI be used or disclosed for this purpose?  
Security question: How is the ePHI protected while stored, used, and  
transmitted?  
Breach question: What facts must be preserved and evaluated after an  
incident?
```

The rules interact. Minimum necessary under the Privacy Rule influences access-management decisions under the Security Rule. Audit evidence generated for security can become critical during breach analysis. De-identification can remove information from PHI status under the Privacy Rule when the regulatory standard is actually met.

Exercise: Classify the question

For each, write Privacy, Security, Breach, or More Than One: “Can we send this report to an AI vendor?”, “How do we encrypt the connection?”, “Who saw the record after a compromised account?”, “Can research receive exact dates?”

Chapter 5 How to Read HIPAA Like an Engineer

Learning objective. Turn a regulation into a question your architecture, code, operations, and evidence must answer.

HIPAA is deliberately technology-neutral. That vagueness is not an excuse to stop at policy language. Read each requirement as a design question. When §164.312(b) calls for

mechanisms to record and examine activity in systems that contain or use ePHI, the engineering question is not “do we have logs?” It is “which actions are recorded, where, with what fields, who can alter them, how are they reviewed, and what proves that coverage is complete?”

The seven-step translation

- 1. Read the standard and identify whether an implementation specification is required or addressable.
- 2. State the security or privacy outcome in plain language.
- 3. Write the developer question the system must answer.
- 4. Name the trust boundary and enforcement point.
- 5. Define the negative test: how would you prove the control cannot be bypassed?
- 6. Define the evidence artifact.
- 7. Record residual risk, owner, and review date.

Addressable does not mean optional

Under the current Security Rule, an addressable implementation specification requires a reasoned determination. Implement it when reasonable and appropriate; use an equivalent alternative when appropriate; or document why the specification is not reasonable and appropriate in the circumstances and how the standard is otherwise met.

Rule	§164.312(e) Transmission security
Developer question	Can unauthorized parties read or modify ePHI in transit?
Architecture answer	TLS/VPN/private connectivity + peer authentication + destination binding + authorization.
Evidence	Configuration, certificate/route records, tests, monitoring, interface inventory.

Mastery check

Why is “we use AES-256” not an answer to an entire HIPAA safeguard?

What is the difference between a control and evidence of a control?

PART II

Control the data

Chapter 6 Minimum Necessary as API and Query Design

Learning objective. Express the Privacy Rule minimum-necessary principle as row scope, field projection, purpose, and data-contract design.

The minimum necessary standard generally requires reasonable steps to limit many uses, disclosures, and requests for PHI to what is needed for the intended purpose. For developers, this is not abstract ethics. It changes selectors, serializers, GraphQL/Convex queries, REST response contracts, exports, background jobs, and model prompts.

Authorization and minimum necessary are related but different

Authorization asks whether the principal may access the resource. Minimum necessary also asks how much of that resource and which fields are needed for the purpose. A clinician may be authorized to access an encounter and still not need every internal note or administrative field for a summary list.

```
# Bad shape: broad row set + broad serializer
Exam.objects.filter(organization=org)
# response includes patient, report body, internal notes, image
metadata...

# Better shape: scope rows, then project the contract
ExamSelector.for_actor(actor, org, facility).values(
    'id', 'exam_type', 'exam_date', 'status'
)[:50]
```

Four questions for every endpoint

- Rows: which records does this actor need?
- Fields: which attributes does this UI or process actually render/use?
- Purpose: what workflow justifies the access?
- Retention/copying: does the consumer keep, cache, export, log, or forward the data?

Client filtering is not minimization

If the server sends the full PHI object and React hides three fields, those fields were still disclosed to the browser. The server-side query/selector and serializer contract must enforce the decision.

Exercise: Shrink the payload

Take an exam-list API that returns the full patient and report. Design a summary contract for the worklist and a separate detail contract for authorized report review.

Chapter 7 De-identification: The Boundary Where PHI Can Stop Being PHI

Learning objective. Explain why de-identification is powerful, strict, and different from pseudonymization or “removing names.”

Under the HIPAA Privacy Rule, health information that is de-identified in accordance with §164.514 is not individually identifiable health information and therefore is not PHI under the Privacy Rule. That is a major boundary change. But it does not mean the data is free of every possible obligation: contracts, research rules, state law, ethical commitments, intellectual-property restrictions, or other privacy regimes can still matter.

HIPAA provides two methods

- Safe Harbor: remove the specified identifier categories and have no actual knowledge that the remaining information could identify the individual.
- Expert Determination: a qualified expert applies accepted statistical/scientific methods, determines that identification risk is very small in the anticipated context, and documents the methods and results.

Pseudonymized is not automatically de-identified

Replacing MRN 12345 with patient_8fd2 does not by itself satisfy Safe Harbor or Expert Determination. A stable code can still permit linkage and can remain PHI depending on how it is created and used.

For medical AI, de-identification is often discussed as a way to reduce exposure outside a HIPAA-regulated relationship. The engineering mistake is to invent a “de-identified” flag without implementing one of the regulatory methods and without testing the transformation boundary.

Mastery check

Why is “remove the patient name” not a de-identification method?

What changes legally when data is properly de-identified under the HIPAA Privacy Rule?

Chapter 8 Safe Harbor in Practice

Learning objective. Implement and test the Safe Harbor method without accidentally retaining dates, location detail, or rare identifying context.

Safe Harbor is mechanical enough to be engineered, but only if the team understands the full rule. The method requires removal of 18 identifier categories for the individual and for relatives, employers, or household members, plus no actual knowledge that the remaining information could identify the individual.

The 18 categories - developer checklist

- Names.
- Geographic subdivisions smaller than a state, subject to the special three-digit ZIP rule.
- All elements of dates directly related to an individual, except year, with special treatment for ages over 89.
- Telephone numbers.
- Fax numbers.
- Email addresses.
- Social Security numbers.
- Medical record numbers.
- Health plan beneficiary numbers.
- Account numbers.
- Certificate/license numbers.
- Vehicle identifiers and serial numbers, including plates.
- Device identifiers and serial numbers.
- Web URLs.
- IP addresses.
- Biometric identifiers such as finger and voice prints.
- Full-face photographs and comparable images.
- Any other unique identifying number, characteristic, or code, subject to the rule's provisions.

Dates, age, and ZIP are where implementations break

Safe Harbor removes date elements more specific than year when the dates are directly related to an individual. Ages over 89 and date elements that indicate such age must be aggregated into a category of age 90 or older. Three-digit ZIP prefixes can be retained only when the geographic unit formed by those prefixes exceeds the population threshold specified by the rule; otherwise the first three digits are replaced with 000.

```
def to_safe_harbor(encounter):
    return {
        'visit_year': encounter.visit_date.year,
        'age_band': '90+' if encounter.age > 89 else
        band(encounter.age),
        'zip3': allowed_zip3(encounter.zip) or '000',
        'icd10': [d.code for d in encounter.diagnoses],
        # no narrative note crosses this boundary
    }
```

Test the 94-year-old

A de-identification test suite should contain adversarial fixtures: age 94, a restricted ZIP prefix, multiple direct identifiers, URLs/IPs, device serials, and narrative text. The test should assert what is absent, not merely what is present.

Exercise: Safe Harbor test matrix

Create five records designed to defeat a naive scrubber. Include a 94-year-old, rare condition, exact procedure date, restricted ZIP prefix, and device serial number. Write expected transformed output.

Chapter 9 Expert Determination and the Limited Data Set

Learning objective. Choose between Safe Harbor, Expert Determination, a limited data set, or keeping PHI inside the regulated perimeter.

Expert Determination

Expert Determination is not a developer self-certification. A person with appropriate knowledge and experience in accepted statistical and scientific principles determines that the risk is very small that the anticipated recipient could identify an individual using the data alone or with other reasonably available information, and the methods and results are documented. It can preserve data utility that Safe Harbor would destroy, such as more precise temporal information, when the expert's analysis supports it.

Limited Data Set

A limited data set under §164.514(e) is still PHI. It excludes specified direct identifiers but can retain information such as dates and certain geography. It may be used or disclosed for research, public health, or health care operations when the regulatory conditions are met, including a Data Use Agreement.

Do not collapse these states

Identified PHI ≠ Limited Data Set ≠ De-identified data. A limited data set remains PHI. De-identified data under §164.514(a)-(b) is not PHI under the Privacy Rule.

Need	Identified PHI	Limited Data Set	De-identified
Exact dates for care	Often yes	Not the usual care path	May remove too much
Research with dates/site geography	Possible with appropriate basis	Often useful + DUA	Expert method may preserve more
Public dataset	No	No	Potentially, subject to other law/contract
Model vendor without HIPAA arrangement	High-risk decision	Still PHI	Possible if truly de-identified

Mastery check

Why is a limited data set still PHI?

When could Expert Determination be preferable to Safe Harbor?

Chapter 10 Free Text, Analytics, Research, and AI Model Boundaries

Learning objective. Design outbound data paths so narrative text, derived data, prompts, and model telemetry do not silently escape the intended privacy boundary.

Structured fields are easier to transform because the schema tells you where identifiers live. Free text is different. Notes can contain names, exact dates, ages, providers, employers, family relationships, facilities, rare conditions, addresses, and narrative clues. Regex can remove obvious patterns and still miss the identifying context that matters.

Three safe design postures

- Do not collect or transmit narrative text when the feature does not need it.
- Keep identified text inside the approved PHI/BAA perimeter and apply minimum necessary, authorization, retention, and audit controls.
- Use a validated de-identification process appropriate to the regulatory method and validate its performance, including human review where warranted.

AI boundary record

- Purpose: why does the model need this information?
- Data class: PHI, limited data set, or de-identified?
- Provider relationship: BAA/contract coverage and service scope.
- Training/retention terms: what happens to prompts, outputs, abuse logs, and telemetry?
- Minimum necessary: what fields/text are actually required?
- Output handling: can the output itself contain PHI or become part of the designated record set?
- Human review and failure behavior: what happens when the model is wrong or unavailable?

Lumen example

A vascular report sent to an AI assistant may contain patient identifiers in headers and identifying details in narrative text. Removing the header name is not enough. Decide first whether the provider may receive PHI. If not, use a real de-identification strategy rather than a cosmetic scrubber.

Exercise: AI data-flow review

For a “discuss this vascular protocol/report” feature, document purpose, input fields, de-identification state, model vendor, retention, logging, output destination, authorization, and failure behavior.

PART III

Build the Security Rule into architecture

Chapter 11 Risk Analysis and Risk Management

Learning objective. Build a defensible risk register tied to real ePHI assets, threats, vulnerabilities, controls, owners, dates, and evidence.

The Security Rule’s risk-analysis requirement is not a vulnerability scan. A scan can find technical weaknesses, but it cannot discover every workflow, vendor dependency, support process, clinical failure mode, or human action. Risk analysis is a system model: what ePHI exists, where it is, what can threaten it, what weaknesses make harm plausible, and what controls reduce the risk.

Write risk scenarios, not nouns

“SQL injection” is a weakness category. A risk scenario is more useful: “An unauthenticated attacker injects a crafted filter value into the report-search path, causing the database to return ePHI from multiple organizations.” The scenario identifies actor, action, vulnerable boundary, asset, and consequence.

```
Risk: Cross-tenant exam disclosure
Asset: patient exam + report
Threat: authenticated user changes exam identifier
Vulnerability: object lookup not tenant-scoped
Control: centralized selector + permission + object ownership check
Evidence: negative API test returns 404/403 and audit event
Owner: reports team
Review: quarterly / material architecture change
```

Exercise: Top ten risks

Write the ten most credible ePHI risk scenarios for your application. Include at least one confidentiality, integrity, availability, insider, vendor, and recovery scenario.

Chapter 12 Administrative and Physical Safeguards Developers Still Need to Understand

Learning objective. Recognize the non-code safeguards that change product requirements and operational design.

Developers often jump directly to §164.312 technical safeguards and miss that the Security Rule also requires administrative and physical safeguards. These requirements create product constraints: workforce authorization changes require timely access changes; incident procedures require usable logs; contingency planning requires recoverable systems; device/media controls affect local downloads and removable media; facility/workstation safeguards affect kiosk and shared-workstation behavior.

Developer touchpoints

- Security management process → risk register, remediation workflow, control ownership.
- Workforce security / termination → provisioning, deprovisioning, session revocation, group synchronization.
- Security awareness → phishing-resistant authentication, warning flows, safe defaults.
- Contingency planning → backups, recovery, emergency-mode operation, data reconciliation.
- Workstation security → session timeout, local-cache policy, download controls, screen exposure.
- Device/media controls → encrypted storage, remote wipe where applicable, disposal and reuse processes.

Architecture is socio-technical

If your application assumes “only trained users will do that,” the training is part of the control. If the consequence is serious, add technical prevention or detection rather than depending on memory alone.

Mastery check

Name one administrative safeguard that directly changes software behavior.

Why can a secure cloud backend still be undermined by workstation or download behavior?

Chapter 13 Access Control and Emergency Access

Learning objective. Turn authorized access into explicit tenant-, facility-, role-, object-, and workflow-scoped decisions, including controlled emergency access.

Authentication answers who presented a credential. Authorization answers whether that principal may perform this action on this resource in this scope now. In a medical application, authorization usually combines identity, active membership, organization, facility, role/capability, object ownership, clinical/workflow state, and sometimes purpose.

```
request
  ↓ identity validation
  ↓ active organization membership
  ↓ facility scope
  ↓ role/capability
  ↓ object belongs to allowed scope
  ↓ workflow-state rule
  ↓ action allowed
  ↓ audit event
```

Emergency access / break-glass

Emergency access is not a backdoor account. Design it as a narrowly controlled capability: explicit invocation, strong authentication, limited scope, short duration, reason capture where appropriate, high-visibility audit, alerting, and post-event review. The exact implementation depends on operational needs and risk analysis.

Lumen pattern

A technologist at Facility A should not be able to fetch Facility B exams by changing an integer in the URL. A support engineer should not become a hidden super-user merely because they can reach an admin console.

Exercise: Authorization matrix

For create exam, edit measurement, finalize, unfinalize, addendum, export PDF, retrieve image, view audit history, and emergency access, define who may act and under what organization/facility/workflow conditions.

Chapter 14 Authentication, Sessions, and Identity Proof

Learning objective. Design authentication as a lifecycle: credential validation, session creation, refresh, revocation, replay resistance, reauthentication, and termination.

A valid JWT is evidence, not authorization. Authentication controls must also handle token lifetime, issuer/audience/signature validation, session state, stolen refresh tokens, logout, replay, password/MFA changes, account disablement, and compromised devices.

Gait example - why session state matters

A robust hosted identity service can rotate refresh tokens, hash stored refresh credentials, detect replay across a token family, bind access tokens to server-side sessions, and revoke sessions after logout or compromise. The application consuming the token must still validate it and apply its own business authorization.

- Reject expired, wrong-issuer, wrong-audience, or invalid-signature tokens.
- Do not store raw refresh tokens server-side when a secure hash can be used for comparison.
- Serialize or otherwise control refresh behavior across browser tabs so rotation is not raced accidentally.
- Revoke server-side sessions after logout, replay, administrative disablement, or other security events.
- Require stronger or fresh authentication for especially sensitive actions when risk analysis supports it.

Mastery check

Why does a JWT not prove the user can open a particular exam?

What should happen to active sessions after refresh-token replay is detected?

Chapter 15 Audit Controls: Security Telemetry vs ePHI Activity

Learning objective. Design audit mechanisms that can reconstruct ePHI activity without turning the logs into another uncontrolled copy of the medical record.

Section 164.312(b) requires mechanisms to record and examine activity in systems that contain or use ePHI. The common mistake is to point at a security-event pipeline and declare the requirement solved. Security telemetry and ePHI activity auditing answer different questions.

Security event log:

"Was there an attack, anomaly, denial, rate-limit hit, CSRF failure?"

ePHI activity log:

"Who viewed/changed/exported which protected resource, when, in what scope, and with what outcome?"

These logs can share infrastructure, but the event catalog, coverage expectations, retention, review process, and access restrictions are different. An authorization-denied event is security telemetry. An authorized physician opening an exam may be ordinary ePHI activity that still matters for reconstruction.

Minimum useful audit fields

- Actor or authenticated subject reference.
- Organization/facility scope.
- Protected resource type and stable reference.
- Action: view, create, edit, finalize, export, send, retrieve media, etc.
- Timestamp and request/correlation identifier.
- Outcome and reason code.
- Session/context information appropriate to risk.
- Destination for disclosures/transmissions when applicable.

What not to log

- Raw JWTs or refresh tokens.
- Passwords or MFA secrets.
- Full report bodies merely to prove they were viewed.
- Long-lived signed URLs or credentials.
- Unnecessary patient names in free-form log messages.

Important legal distinction

Security audit controls support investigations and may support disclosure-accounting workflows where applicable, but §164.312(b) itself is not the same thing as the Privacy Rule's accounting-of-disclosures right. Do not equate "every read" with a disclosure-accounting entry without analyzing the applicable rule.

Rule	§164.312(b)
------	-------------

Developer question	Can we reconstruct meaningful activity in systems that contain/use ePHI?
Architecture answer	Central event catalog + durable audit store + protected access + review/alerting.
Evidence	Coverage tests, sample events, retention config, access reviews, incident reconstruction exercises.

Exercise: Audit coverage map

List every PHI-bearing read/write/export/send path. Mark the event emitted, required fields, whether users can alter it, retention, and who reviews it.

Chapter 16 Integrity and Clinical Meaning

Learning objective. Protect ePHI from improper alteration or destruction and preserve the clinical meaning of the record across calculations, signing, correction, and transport.

Integrity is not merely “the database transaction committed.” In clinical software, the record can be technically consistent and still be clinically wrong: the wrong patient may be matched, the wrong criteria version may be applied, a finalized report may be silently overwritten, or an HL7 ORU may be routed to the wrong encounter.

Integrity controls by workflow stage

- Template → exam: record template/protocol version and reproducibly create expected segments/measurements.
- Measurements → calculations: deterministic reviewed logic with threshold-boundary tests.
- Conclusion: provenance for rule version, human edits, and overrides.
- Finalize/sign: workflow lock, authorization, immutable event history, controlled addendum/correction.
- HL7/DICOM: patient/order matching, message/study identifiers, destination binding, ACK/reconciliation.
- Storage: database constraints, backups, version history, checksums/signatures where risk analysis justifies them.

Do not confuse these controls

Validation $\neq$ integrity proof. Transactionality $\neq$ auditability. Backup $\neq$ tamper detection.
 Authorization $\neq$ evidence that a record was unchanged.

Exercise: Wrong-patient scenario

A signed vascular report is correct in Lumen but arrives attached to the wrong patient in the EHR. Identify every possible boundary failure and the evidence needed to reconstruct the event.

Chapter 17 Transmission Security and Encryption Decisions

Learning objective. Choose protection based on the real path, peer, network, data, and threat - not merely the protocol name.

The current Security Rule is technology-neutral. Encryption implementation specifications are addressable under the current rule, which means the organization must perform the required reasoned analysis. In modern internet/cloud designs, strong encryption in transit is ordinarily a baseline expectation, but the legal analysis and the architecture should still be documented rather than reduced to “TLS = compliant.”

Every connection record should answer

- Source workload and owner.
- Destination and owner.
- Data classification.
- Protocol and port.
- Encryption or protected network.
- Peer authentication.
- Authorization and allowed direction.
- Timeout/retry/replay behavior.
- Monitoring/alerting.
- Downtime and fail-safe behavior.

HL7 example

HL7 defines message structure. MLLP frames a message. TCP moves ordered bytes. None of those alone provides confidentiality, peer identity, or application authorization. Those protections come from the surrounding network and security architecture.

Mastery check

Why is “MLLP over TCP” not a transmission-security answer?

What evidence would prove that an API path is protected in transit?

Chapter 18 Availability, Backups, and Downtime

Learning objective. Define recovery in measurable clinical terms and prove it with restoration and reconciliation exercises.

Availability means authorized users can obtain ePHI when needed. A backup is not a recovery capability until it has been restored successfully. Medical applications also need safe degraded behavior: what happens to studies in progress, queued HL7 results, signed reports, identity dependencies, and image retrieval when one component is unavailable?

Define the recovery contract

- RPO: how much data loss is tolerable?
- RTO: how long can the capability be unavailable?
- Restoration order and dependencies.
- Immutable/offline or otherwise resilient backup strategy appropriate to risk.
- Downtime workflow for clinicians and technologists.
- Queue preservation and idempotent replay.
- Validation after restoration.
- Reconciliation of work created during the outage.

Exercise: Database outage tabletop

At 10:15 a.m. the primary database is unavailable while two physicians are signing studies and 37 HL7 results are queued. Write the first five decisions, the degraded behavior, evidence preserved, recovery order, and reconciliation steps.

Chapter 19 Documentation, Evidence, Retention, and Review

Learning objective. Build a control evidence system so safeguards remain inspectable over time rather than existing only in developer memory.

The Security Rule includes policies/procedures and documentation requirements. HHS summarizes a six-year retention rule for required documentation, measured from creation or the date last in effect, whichever is later. Developers should therefore design evidence as a durable artifact: risk decisions, control configurations, diagrams, access reviews, restore tests, incident records, and change history.

Evidence packet for a control

```
Control: cross-tenant authorization
Owner: application security / reports
Design: selector + permission + object ownership
Code reference: current repository path + commit
Tests: allowed + denied + ID-tampering + concurrency
Runtime evidence: denied audit event samples
Review: date / reviewer / findings
Residual risk: documented
Next review: date or trigger
```

Evidence must age well

A screenshot of a green CI run is weak evidence if nobody can tell which commit, environment, configuration, or test suite it represents. Prefer traceable artifacts tied to version, owner, scope, and date.

Exercise: Build a control card

Choose one safeguard in your application and create an evidence packet with owner, design, implementation, tests, runtime evidence, residual risk, and review trigger.

PART IV

Clinical systems and cloud boundaries

Chapter 20 HL7, MLLP, TCP, and Interface Security

Learning objective. Understand the transport stack well enough to secure clinical messaging and diagnose failures without confusing message semantics with network protection.

Think in layers. An HL7 v2 ORU message carries clinical meaning. MLLP adds delimiters so a receiver can find message boundaries on a byte stream. TCP gives reliable ordered delivery between sockets. The operating system's networking stack moves the bytes. Security must be intentionally added around this chain.

```

Lumen ORU
  ↓ HL7 v2 message
MLLP framing
  ↓
TCP socket
  ↓
TLS / VPN / private route / network controls (design-specific)
  ↓
Hospital interface engine
  ↓ ACK / NACK
  ↓ reconciliation + audit

```

Interface controls that matter

- Fixed and verified destination; do not let untrusted client input choose the EHR endpoint.
- Network segmentation and allowlisting appropriate to the deployment.
- Protected transport and peer authentication appropriate to the route.
- Message control IDs and duplicate/replay strategy.
- ACK handling, timeout policy, retry limits, dead-letter/reconciliation workflow.
- Patient/order matching before and after transport.
- Audit events that do not dump entire messages into ordinary logs.

Mastery check

What security property does TCP provide that HL7 does not?

What does an ACK prove - and what does it not prove?

Chapter 21 DICOM and Medical Image Security

Learning objective. Treat medical images and imaging metadata as first-class ePHI with controlled ingestion, storage, retrieval, sharing, and deletion.

DICOM is not “just image files.” Studies can contain patient identifiers in structured metadata, burned-in annotations, device information, dates, accession numbers, and other clinical context. Security must cover both pixel data and metadata.

Questions for every DICOM path

- How is the sending node/workload identified?
- How is the patient/study matched?
- Is transport protected for DIMSE or DICOMweb?
- Where are objects stored and encrypted?
- Who may retrieve a study and how is tenant/facility scope enforced?
- How are presigned URLs scoped and expired?
- Are retrievals audited?
- How are thumbnails, caches, temp files, exports, and local downloads handled?

Lumen media-service boundary

Authorization must happen before issuing a signed object-storage URL. Object-key secrecy is not authorization. A guessed key must still fail because the application verifies organization, facility, resource ownership, and actor permission before presigning.

Exercise: Trace one image

Trace a DICOM object from scanner/import through media service, object storage, browser retrieval, cache, and deletion. Mark every copy and temporary representation.

Chapter 22 Cloud Vendors, BAAs, Secrets, and Object Storage

Learning objective. Evaluate each dependency as part of the regulated system and keep contractual scope aligned with actual technical data flow.

A cloud provider’s HIPAA program does not transfer compliance to your application. Determine which services are covered by contractual assurances, which resources actually receive ePHI, how identities and keys are managed, and how data is retained, backed up, logged, and deleted.

Vendor inventory fields

- Vendor/service
- Purpose
- Data received or maintained
- PHI/ePHI?
- BAA/contract status
- Regions/services in scope
- Encryption/key ownership
- Retention/deletion
- Subprocessors
- Security evidence owner

Secrets

Secrets should not live in source code, client bundles, ordinary logs, support tickets, or screenshots. Centralize secret management, minimize scope, rotate when compromise or policy requires, and design services so one credential does not grant unrestricted access across tenants or environments.

Object storage

Keep buckets private by default. Authorize at the application boundary before issuing short-lived access. Bind object keys to server-side resource truth. Audit grants and retrieves appropriate to risk. Test cross-tenant key substitution.

Mastery check

Why is “the vendor signs a BAA” not sufficient technical evidence?

What is the security difference between a private bucket and authorized object access?

Chapter 23 Multi-tenancy, Facilities, Support Access, and Break-Glass

Learning objective. Prevent one customer, facility, support user, or automation path from becoming a hidden cross-tenant authorization bypass.

Multi-tenant medical software adds a second axis beyond ordinary role-based access: every read and write must be scoped to the correct tenant, and often a facility. The most

dangerous bugs occur when one endpoint applies tenant scope while another uses an unscoped primary-key lookup.

Layered pattern

- Models store organization ownership and integrity constraints.
- Selectors apply organization/facility filters to reads.
- Services enforce authorized writes and workflow invariants.
- Permissions reject unauthorized actions before protected logic runs.
- Serializers expose only approved fields.
- Views orchestrate but never trust client-supplied tenant truth.
- Tests attempt horizontal and vertical privilege escalation.

Support access

Support impersonation is high risk because it can bypass normal accountability. Prefer diagnostic tooling that exposes necessary technical state without becoming the user. If impersonation or break-glass access is truly needed, constrain it with approval, visibility, expiry, scope, strong audit, and post-use review.

Exercise: IDOR hunt

List every endpoint that accepts an exam ID, report ID, media ID, facility ID, or organization slug. For each, identify where server-side ownership is verified and write the cross-tenant negative test.

PART V

Secure software engineering

Chapter 24 OWASP Top 10:2025 for Medical Applications

Learning objective. Use the OWASP Top 10 as a risk-awareness map while recognizing that it is not a complete application-security verification standard.

OWASP Top 10:2025 is an awareness document for critical web-application risks. The 2025 list is: Broken Access Control; Security Misconfiguration; Software Supply Chain Failures; Cryptographic Failures; Injection; Insecure Design; Authentication Failures; Software or Data Integrity Failures; Security Logging and Alerting Failures; and Mishandling of Exceptional Conditions. OWASP explicitly recommends ASVS when you need a verifiable security standard rather than a Top 10 awareness list.

2025 category	Medical application failure example
A01 Broken Access Control	Tenant/facility/object bypass, IDOR, role escalation, support backdoors.
A02 Security Misconfiguration	Debug mode, permissive CORS, public buckets, weak headers, default credentials, unsafe cloud/IAM settings.
A03 Software Supply Chain Failures	Compromised dependencies, build systems, CI actions, registries, package confusion, unverified artifacts.
A04 Cryptographic Failures	Cleartext PHI, poor key handling, obsolete protocols, incorrect certificate verification.
A05 Injection	SQL/command/template/LDAP/XSS and parser injection across APIs, search, HL7, files, admin tooling.
A06 Insecure Design	Missing threat model, unsafe workflow assumptions, no abuse cases, no trust-boundary design.
A07 Authentication Failures	Weak session lifecycle, token replay, broken MFA/reset, account enumeration.
A08 Software or Data Integrity Failures	Untrusted updates/artifacts, unsigned data/code paths, unsafe deserialization, broken provenance.
A09 Security Logging and Alerting Failures	No audit coverage, no alerts, noisy logs, secrets/PHI in logs, no review.
A10 Mishandling of Exceptional Conditions	Fail-open auth, swallowed errors, unsafe retries, inconsistent partial state, resource exhaustion.

HIPAA + OWASP

HIPAA asks what safeguards a regulated health-information system must have. OWASP asks how modern applications commonly fail. A secure medical application needs both legal/regulatory analysis and rigorous application-security engineering.

Exercise: Top 10 mapping

For each OWASP 2025 category, identify one control in your application and one missing negative test or evidence artifact.

Chapter 25 OWASP ASVS 5.0 as a Verification Backbone

Learning objective. Use ASVS 5.0 to turn broad security intentions into testable application-security requirements.

OWASP ASVS 5.0 is a better backbone for verification than claiming “OWASP Top 10 compliance.” OWASP describes ASVS as a basis for testing application technical security controls and the technical controls the application relies upon. Use it to define what must be verified, then map relevant requirements to HIPAA risks and your architecture.

A practical mapping approach

1. Choose the assurance level appropriate to the application's sensitivity and risk. Medical applications often justify a stronger verification posture than ordinary low-risk sites.
2. Select ASVS requirements relevant to your architecture and deployment.
3. Map each requirement to a code/configuration enforcement point.
4. Add automated tests where possible and manual verification where necessary.
5. Retain evidence with version, environment, owner, and date.
6. Reassess after material architecture, identity, integration, or vendor changes.

Do not market a checklist as proof

OWASP itself discourages claims of complete Top 10 coverage. A mature statement is narrower: “Our secure development and verification program uses OWASP Top 10:2025 for risk awareness and ASVS 5.0 as a verification reference,” followed by evidence.

Exercise: Build an ASVS control map

Choose 15 ASVS requirements relevant to authentication, authorization, sessions, data protection, logging, APIs, configuration, and secure communication. Map each to implementation and test evidence.

Chapter 26 Threat Modeling and Secure Design

Learning objective. Find architectural security failures before they become code by modeling assets, trust boundaries, attackers, abuse cases, and safety consequences.

Insecure design cannot be patched away with input validation. Threat modeling asks what the system is, where trust changes, which assets matter, who may attack or misuse it, and what controls should exist before implementation.

Threat-model one feature before coding

- Asset: what PHI, capability, or clinical decision needs protection?
- Actors: patient, clinician, technologist, admin, support, attacker, service account, vendor.
- Trust boundaries: browser/API, API/database, service-to-service, cloud/vendor, hospital network, AI provider.
- Abuse cases: ID tampering, replay, role escalation, wrong-patient mapping, mass export, poisoned input, denial of service.
- Safety behavior: what happens when the dependency is wrong, slow, unavailable, or malicious?
- Evidence: which test proves the design assumption?

Medical design includes clinical misuse

A system can be secure from attackers and still be unsafe if the wrong patient is selected, a stale criteria version drives a conclusion, or a retry sends duplicate results. Treat clinical workflow invariants as security-relevant design constraints.

Exercise: Threat model media presign

Threat-model the path from browser request to authorization to object-storage signed URL. Include cross-tenant guessing, URL leakage, expired credentials, support access, logs, and replay.

Chapter 27 Secure Development Lifecycle and Supply-Chain Assurance

Learning objective. Embed security into design, coding, review, dependency management, CI/CD, deployment, and maintenance.

Secure development is a lifecycle, not a final penetration test. The 2025 OWASP list makes software supply-chain failures a first-class category because modern applications trust package registries, CI actions, build images, lockfiles, artifact stores, deployment tokens, and third-party code.

Engineering habits by phase

- Design: data classification, threat model, trust-boundary diagram, privacy purpose/minimum necessary.
- Build: centralized authorization, safe parsing, parameterized queries, secret management, secure defaults.
- Review: security-context code review, dependency changes, config/IAM changes, schema/data-flow changes.
- Test: positive/negative auth, concurrency, fuzzing where useful, integration failure, recovery, log redaction.
- CI: pinned/controlled dependencies and actions, SCA, secret scanning, test gates, artifact integrity.
- Deploy: least-privilege deployment identities, immutable artifacts, approvals, rollback, environment separation.
- Operate: patching, vulnerability triage, alerts, incident response, restore drills, periodic access and risk review.

Gait principle

Intelligence does not grant authority. An AI agent can investigate, propose, or validate, but consequential deployment or security actions must remain constrained by deterministic policy, capability boundaries, immutable artifacts, and human approval where required.

Mastery check

Why can a clean source repository still produce a compromised deployment?

What supply-chain evidence would you retain for a production release?

Chapter 28 Testing Controls, Exceptional Conditions, and Fail-Safe Behavior

Learning objective. Prove controls under allowed, denied, concurrent, partial-failure, timeout, retry, and dependency-outage conditions.

Happy-path tests prove functionality. Security tests prove boundaries. The 2025 OWASP A10 category is a useful reminder that systems often fail during abnormal conditions: a timeout skips an authorization check, a retry duplicates a clinical message, an exception returns raw PHI, or a downstream outage leaves a workflow in an unsafe intermediate state.

A security test matrix

- Allowed actor / allowed resource.
- Allowed actor / wrong tenant.
- Allowed actor / wrong facility.
- Wrong role / correct object.
- Expired/revoked session.
- Concurrent single-use action.
- Dependency timeout before commit.
- Dependency timeout after commit.
- Retry after uncertain outcome.
- Malformed clinical payload.
- Logging path fails.
- Audit store unavailable.
- Object store unavailable.
- Identity provider unavailable.
- HL7 receiver returns NACK or never ACKs.

Fail closed is not universally “stop everything”

In healthcare, availability and patient safety matter. The design question is safe failure: which actions must deny, which can continue in degraded mode, how data is queued, what is visible to users, and how reconciliation occurs afterward.

Exercise: Break the workflow

Choose report finalization or HL7 send. Write tests for timeout before commit, timeout after commit, duplicate retry, audit-write failure, permission change mid-request, and downstream NACK.

Chapter 29 Incident Response, Breach Readiness, and Evidence Preservation

Learning objective. Design the product so responders can contain, investigate, recover, and provide reliable facts for privacy/legal assessment.

An incident is an operational event requiring response. A breach is a legal determination under the HIPAA Breach Notification Rule. Developers should preserve trustworthy facts rather than making that legal conclusion alone.

First-hour engineering responsibilities

- Contain the vulnerable capability without destroying evidence.
- Preserve relevant logs, deployments, database state, identity/session evidence, and configuration.
- Determine earliest plausible exposure and affected resources/tenants.
- Revoke or rotate compromised credentials and sessions when appropriate.
- Coordinate with privacy/security/legal leadership according to the incident plan.
- Repair the central control, not merely the single observed URL.
- Add regression tests and reassess similar paths.

Lumen scenario

A technologist changes an exam ID and sees another organization's exam. Treat it as a possible systemic object-authorization failure until proven otherwise. Search all resource lookup paths that use the same pattern; do not patch only the reported endpoint.

Exercise: Cross-tenant first hour

Write the first-hour response to a report that one user viewed another organization's exam. Include containment, evidence, scope questions, credential actions, communication, remediation, and verification.

PART VI

Capstone

Chapter 30 Perform a HIPAA Engineering Review of Lumen - Then Your Own System

Learning objective. Produce a complete, evidence-driven architecture review that integrates privacy, HIPAA safeguards, clinical workflow, OWASP risks, verification, vendors, and recovery.

Step 1 - System statement

In 200 words, describe the product, users, regulated customers, PHI/ePHI, integrations, deployment, vendors, and trust boundaries. If you cannot state the system clearly, you cannot analyze it clearly.

Step 2 - Data inventory and purpose

Data	PHI/ePHI?	Purpose	System of record	Copies/destinations	Retention owner

Step 3 - Trust-boundary map

Draw every browser, API, service, database, object store, cache, queue, identity provider, observability provider, developer/support surface, AI provider, backup, EHR/interface engine, DICOM node, export, and temporary file location. Mark each arrow with data type, protocol, authentication, authorization, and transport protection.

Step 4 - HIPAA safeguard map

For every applicable safeguard, complete: Rule → Developer question → Architecture → Code/config enforcement → Negative test → Evidence → Gap → Owner → Review date.

Step 5 - Privacy engineering

- Identify where minimum necessary is enforced in query/selector and response contracts.
- Identify any de-identification boundary and the regulatory method used.
- Identify limited data sets and DUAs, if any.
- Identify every AI/analytics/research outbound path and its legal/contractual basis.

Step 6 - OWASP / ASVS verification

Map OWASP Top 10:2025 risks to the application. Use ASVS 5.0 requirements as the verifiable control baseline appropriate to the application. Record evidence, gaps, and exceptions.

Step 7 - Recovery and incident proof

- Perform a restore test.
- Run an identity-provider outage tabletop.
- Run an HL7 receiver outage/retry exercise.
- Run a cross-tenant access incident tabletop.
- Verify audit coverage and log redaction.

Step 8 - Go-live evidence packet

- Current system/data-flow diagrams.
- Risk analysis and treatment plan.
- Privacy/minimum-necessary/de-identification decisions.
- Authorization matrix and tenant-isolation tests.
- Authentication/session lifecycle evidence.
- Audit event catalog and coverage tests.
- Integrity/workflow-state tests.
- Encryption/network/interface records.
- Backup restore and downtime exercises.
- Vendor inventory and agreements.
- OWASP/ASVS verification results.
- Incident response plan and exercise records.
- Deployment, rollback, and supply-chain evidence.
- Named owners and review dates.

The standard you should aim for

You can point to any important claim - “only this facility can read this study,” “the report cannot be silently changed after signing,” “this dataset is Safe Harbor de-identified,” “this HL7 destination is fixed,” “we can restore within our RTO” - and show the architecture, code/configuration, negative test, and evidence that support it.

Appendix A Quick Reference: HIPAA Developer Questions

Area	Question
Scope	What PHI/ePHI exists? Why is it in scope? Which entity/BA relationship creates the obligation?
Purpose	Why is the data used/disclosed? Does minimum necessary apply?
Identity	How is the actor authenticated and how is session state revoked?
Authorization	How are tenant, facility, role, object, workflow state, and emergency access enforced?
Audit	Can we reconstruct access, changes, exports, sends, denials, and security events without logging excess PHI?
Integrity	How do we detect/prevent improper alteration or destruction and preserve clinical provenance?
Transmission	How are peers identified and ePHI protected across every network boundary?
Availability	What are RPO/RTO, downtime behavior, restoration order, and reconciliation?
Vendors	Which vendors can receive/maintain ePHI and what contractual/service scope applies?
AI	Is the input PHI, limited data set, or de-identified? What happens to prompts/outputs/telemetry?
Evidence	What test, config, review, event, or exercise proves the control works?

Appendix B OWASP Top 10:2025 Checklist

☐ **A01 Broken Access Control**

Tenant/facility/object bypass, IDOR, role escalation, support backdoors.

Control/evidence:

☐ **A02 Security Misconfiguration**

Debug mode, permissive CORS, public buckets, weak headers, default credentials, unsafe cloud/IAM settings.

Control/evidence:

☐ **A03 Software Supply Chain Failures**

Compromised dependencies, build systems, CI actions, registries, package confusion, unverified artifacts.

Control/evidence:

☐ **A04 Cryptographic Failures**

Cleartext PHI, poor key handling, obsolete protocols, incorrect certificate verification.

Control/evidence:

☐ **A05 Injection**

SQL/command/template/LDAP/XSS and parser injection across APIs, search, HL7, files, admin tooling.

Control/evidence:

☐ **A06 Insecure Design**

Missing threat model, unsafe workflow assumptions, no abuse cases, no trust-boundary design.

Control/evidence:

☐ **A07 Authentication Failures**

Weak session lifecycle, token replay, broken MFA/reset, account enumeration.

Control/evidence:

☐ **A08 Software or Data Integrity Failures**

Untrusted updates/artifacts, unsigned data/code paths, unsafe deserialization, broken provenance.

Control/evidence:

☐ **A09 Security Logging and Alerting Failures**

No audit coverage, no alerts, noisy logs, secrets/PHI in logs, no review.

Control/evidence:

☐ **A10 Mishandling of Exceptional Conditions**

Fail-open auth, swallowed errors, unsafe retries, inconsistent partial state, resource exhaustion.

Control/evidence:

Appendix C Vocabulary for Medical Application Developers

BAA — Business Associate Agreement.

CE — Covered Entity.

PHI — Protected Health Information.

ePHI — Electronic Protected Health Information.

DUA — Data Use Agreement used with a limited data set under the applicable Privacy Rule provisions.

OCR — HHS Office for Civil Rights, which administers and enforces HIPAA privacy/security rules.

RPO — Recovery Point Objective: tolerated data-loss window.

RTO — Recovery Time Objective: target time to restore a capability.

HL7 v2 — Clinical messaging standard commonly used for ADT, orders, results, and other interfaces.

MLLP — Minimal Lower Layer Protocol, a framing convention commonly used to carry HL7 v2 messages over a stream such as TCP.

TCP — Transport protocol providing reliable ordered byte delivery between endpoints.

ACK/NACK — Acknowledgment / negative acknowledgment used in message-processing workflows.

DICOM — Standard for medical imaging information and communication.

JWT — JSON Web Token; a token format, not an authorization system by itself.

IDOR/BOLA — Object-authorization weakness where changing an identifier can expose another resource.

ASVS — OWASP Application Security Verification Standard.

SCA — Software Composition Analysis for dependency risk.

SBOM — Software Bill of Materials: inventory of software components.

Reference Map

Validate production decisions against current primary sources. The references below were used to shape this September 2026 edition.

HHS - The Security Rule

<https://www.hhs.gov/hipaa/for-professionals/security/index.html>

HHS - Summary of the HIPAA Security Rule

<https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html>

eCFR - 45 CFR Part 164

<https://www.ecfr.gov/current/title-45/subtitle-A/subchapter-C/part-164>

HHS - Summary of the HIPAA Privacy Rule

<https://www.hhs.gov/hipaa/for-professionals/privacy/laws-regulations/index.html>

HHS - Minimum Necessary Requirement

<https://www.hhs.gov/hipaa/for-professionals/privacy/guidance/minimum-necessary-requirement/index.html>

HHS - De-identification Guidance

<https://www.hhs.gov/hipaa/for-professionals/special-topics/de-identification/index.html>

HHS - Limited Data Set / Research guidance

<https://www.hhs.gov/hipaa/for-professionals/privacy/guidance/research/index.html>

HHS - Security Rule NPRM

<https://www.hhs.gov/hipaa/for-professionals/security/hipaa-security-rule-nprm/index.html>

OWASP Top 10:2025

<https://top10.owasp.org/2025/>

OWASP Application Security Verification Standard 5.0

<https://owasp.org/projects/asvs>

OWASP Web Security Testing Guide

<https://wstg.owasp.org/>

Regulatory freshness

This book deliberately distinguishes the Security Rule currently in effect from HHS's proposed cybersecurity modifications. Re-check HHS/OCR and eCFR for final-rule changes before relying on a specific implementation specification.

Final Self-Assessment

I can now...	Not yet	With help	Independently
Explain covered entity vs business associate and draw the BAA perimeter.	☐	☐	☐
Classify PHI/ePHI and explain why context and linkage matter.	☐	☐	☐
Design minimum-necessary APIs and query projections.	☐	☐	☐
Explain Safe Harbor, Expert Determination, and Limited Data Sets.	☐	☐	☐
Review AI/model data flows for PHI, contracts, minimization, retention, and logging.	☐	☐	☐
Map a complete clinical data flow and trust boundaries.	☐	☐	☐
Write a useful HIPAA risk scenario and treatment plan.	☐	☐	☐
Review tenant/facility authorization and emergency access.	☐	☐	☐
Design audit events without copying excessive PHI into logs.	☐	☐	☐
Explain clinical integrity controls for signing, addenda, HL7, and DICOM.	☐	☐	☐
Explain HL7 + MLLP + TCP + transport protection.	☐	☐	☐
Define RPO/RTO and prove restoration/reconciliation.	☐	☐	☐
Map OWASP Top 10:2025 and use ASVS 5.0 as a verification reference.	☐	☐	☐
Run a cross-tenant incident tabletop and preserve evidence.	☐	☐	☐
Assemble evidence instead of merely claiming compliance.	☐	☐	☐